

Foundations of scientific programming

Monday 7 September 2026 — lecture and three labs.



MORNING	AFTERNOON	FORMAT	LABS
8h30 – 11h45	13h30 – 16h45	6 pairs	3

WHY YOU ARE HERE

This day is not for this course unit

It is for **the other ones.**

Remote sensing

Data analytics

...

Your three engineering projects

Your placement in March

Your first job

Everything we do today, you will be using within a fortnight — in other modules, projects, internships, or jobs.

Two aims, and they are different

1 • Understand what programming *is*

What a language is. What happens when you press run. Why there are several, and what actually separates them.

The test: by 11h45 you can open a script you did not write and follow what it does.

2 • How you *work* when your work involves *code*

Where you put files. What you call them. How you find out why something broke.

This half you will still be using in ten years, whatever language you end up writing.

Nobody writes a model by tonight. The point is not to produce code — it is to stop being a stranger to it.

Why write code instead of using software that already exists?

A software is a program somebody else wrote. That is its strength — you need to know nothing to use it. And it is its limit: **you can only ever do what its authors imagined.**

FOUR THINGS CLICKING CANNOT DO

1

Scale

A software like Excel stops loading data after 1 048 576 rows. Not a guideline — a wall. A full dataset such as Comtrade runs to tens of millions.

2

Repetition

You spend a day cleaning a dataset by hand. Next year the data is updated. By hand: another day. By script: one filename, four seconds.

3

Traceability

Cleaned by hand, the cleaning exists nowhere. The script **is** the record of what you did. Somebody else can read it. **You can read it**, six months later.

4

Freedom

The model you need does not exist yet. In research, R&D, and engineering, this is the normal case, not the exception.

Public Health England, October 2020

A contact-tracing pipeline that passed through an old Excel format, capped at 65 536 rows.

65 536

For an old Excel format. The row ceiling of a modern Excel sheet is 1 048 576

~16 000

COVID cases dropped, and therefore never contact-traced

0

Warnings displayed. Nothing was shown. Nobody was told.



A script has no such ceiling — and nothing vanishes without telling you.

Things that come up every single year

200	Sentinel tiles to download and clip to the same mask
30	Excel files from thirty people, each with different column headings
3 000	Field photographs to rename from the timestamp inside each file
4 GB	A NetCDF climate file — which Excel cannot open at all, at any size
90 s	To recompute everything at 4 % rate instead of 2 %, because your jury asked

Every one of those: an afternoon of misery by hand, or between five and fifty lines of code.

What it does to you

A consumer of tools

You open QGIS, you click, you get an output, and you trust it. Where everyone starts.



Someone who can audit a model

Read it. Question it. Doubt it. Challenge it. Say what it actually does — instead of believing it.

The difference between **using EFISCEN** and **being able to say what EFISCEN does**. The second is what January asks of you.

Not a coding skill — **a professional posture**. In eighteen months someone will put a model output in front of you and tell you what policy it justifies. Your value at that moment is entirely in **whether you can ask the right question** about how that number was produced.

Do not automate everything

If you do it once, on twenty rows, do it by hand.

A script costs time to write. That cost is paid once and repaid on repetition. No repetition, no repayment: you wasted your time.

The question is never “can this be coded”. It is: will this happen again, and does anyone need to know how I did it? If either answer is yes — write the script.

The reproducibility ladder

4

A stranger, three years from now, gets exactly my numbers

What a journal requires. What forest policy requires.

OCTOBER



3

It runs on my partner's machine

Without calling you. Verified by someone other than you, at 14h25.

TONIGHT

2

It runs on my machine, six months from now

Almost nobody. Structure, relative paths, a README.

LAB 1

1

It runs on my machine, right now

Everyone gets there. Undergraduate lab level.

BEFORE LUNCH

If at any point this year you cannot see why we are doing something, **ask me which rung it is for.**
It is not a device for today. **It is the thread of this entire course unit.**

Pairs, core, challenges

Your pair is already made

Built from the questionnaire, not at random. **Every pair mixes levels** — this cohort runs from never-written-a-line to automating processing chains two years ago.

To the comfortable: explaining is the fastest way to find out you did not quite understand. To those starting at zero: **you are not behind, you are on time.**

These are also your January cartography pairs.

Core and challenges

Every lab session has a **core** everyone finishes and **challenges** to go further.

Finished the core while I am with someone else? **Do not wait for me.** Take a challenge.

Stuck: partner first, me second. I am alone today, morning and afternoon, for twelve people.

Programme

8h55	The language landscape
9h25	A shared minimum vocabulary — reading a script
10h00	Break
10h15	Editors and IDEs · the terminal
11h00	The anatomy of a project — and of a script
11h45	End of the morning
13h30	Three labs: a clean project · dirty data · five broken scripts

Did all three answer on Friday?

```
python --version  
git --version  
quarto --version
```



Hands up if one of them gave you an error. We fix it at 10h00 — not at 13h30. By 13h30 everybody has to be able to work.

PART TWO · 8H55 – 9H25

The language landscape

A wide variety of programming languages. Two axes to sort them by. And one question that separates all of them.

BEFORE I SHOW YOU ANYTHING

Shout out every language you have ever heard of.

Any language. You do not have to know it — you only have to have heard the name.



Some of them are not programming language

HTML, CSS, Markdown, YAML, JSON...

For example, HTML describes what is on a page, while CSS describes what it looks like.

Neither computes anything, neither makes a decision, neither repeats an operation. They **describe**.

You will write description languages this year

Markdown — formatted text. **YAML** — the configuration file in your project template. **JSON, TOML** — data exchange.

A language that **computes** and a language that **describes** are different objects. Worth having the distinction.

And Excel? A spreadsheet with formulas **is** a program — one where you cannot see the code, the logic is scattered over ten thousand cells, and nobody can review it.

When does the translation into machine code happen?

NEVER	Descriptive Nothing is executed. Files, not programs.	HTML · CSS · Markdown · YAML · JSON
BEFORE	Compiled All of it, once, into a binary. Fast — and a build step every time you touch a comma.	C · C++ · Fortran · Rust · Go · Swift
DURING	Interpreted Line by line, and the translation is thrown away each time. You get a console. You pay in speed.	R · Python · PHP · VBA
HALFWAY	Bytecode + virtual machine Compiled ahead of time to a code no real processor understands. A virtual machine finishes the job.	Java · C#
DURING, AND KEPT	JIT — Just In Time Each piece is compiled the first time it is called, and the machine code is kept. First call slow, every later one fast.	Julia · JavaScript · MATLAB

The board, sorted

■ descriptive ■ compiled ■ interpreted ■ bytecode + VM ■ JIT · underlined = specialised, not general-purpose

■ HTML	the content of a web page
■ CSS	how that page looks
■ Markdown	formatted text
■ YAML/JSON	configuration, data exchange
■ Assembly	one line per machine instruction
■ C	operating systems, drivers
■ C++	many ecosystem models
■ <u>Fortran</u>	every climate model, since 1957
■ Rust	C speed, memory safety proved
■ Go	network services
■ Swift	iOS and macOS applications

■ Java	corporate back-ends, Android
■ C#	Microsoft ecosystem, Unity
■ Julia	speed without leaving a high-level language
■ JavaScript	the only language a browser runs
■ <u>MATLAB</u>	numerical work; commercial licence
■ Python	everything, competently
■ <u>R</u>	statistics, graphics, data
■ <u>VBA</u>	Excel macros
■ <u>PHP</u>	the server side of much of the web
■ <u>SQL</u>	asking a database a question — declarative
■ <u>Scratch</u>	the concepts, for children

Three ways to translate a book

Compiled

Translate the entire book once, then sell the translation. Fast to read.

Change one comma in the original and you retranslate everything.

C · C++ · Fortran

Interpreted

A live interpreter beside you, translating sentence by sentence. Change your mind halfway through — no problem.

But she starts from scratch every time.

R · Python

JIT

An interpreter who **writes down each sentence the first time** she translates it, and reads the note back afterwards.

Slow once. Fast for ever after.

Julia · JavaScript

R and Python are interpreted. You type a line, it runs, you see the result — ideal for exploring data, and the reason a heavy simulation is slow in them.

General-purpose or specialised

Specialised

Designed for one domain, and excellent in it. **R for statistics**. SQL for databases. MATLAB for signal processing.

General-purpose

Designed to do everything competently. **Python**. Java. C.

Not a value judgement — a **design decision**, and you will feel its consequences within the hour.

R

Descended from S, Bell Labs, 1970s. Written **by** statisticians **for** statisticians.

Strengths

Statistics, mixed models, graphics (**ggplot2**). Roughly **20 000 packages** on CRAN.

In forestry

Inventory analysis, dendrometry, fitted growth models, spatial work (**sf**, **terra**).

Weakness

Step outside a table and it gets awkward. Rename 3 000 photographs from the timestamp inside each file: fifteen lines of standard library in Python; in R, hunting for a package that half does it.

Python

1991, Guido van Rossum. Designed as a readable general-purpose language.

Strengths

Machine learning (**scikit-learn**, **torch**), automation, image processing — **and being the language every other tool speaks**. QGIS speaks Python. Google Earth Engine speaks Python.

In forestry

Remote sensing, processing chains, anything that means “make two pieces of software talk to each other”.

Weakness

For fine statistical work, R’s ecosystem is still ahead.

Julia, and the two-language problem

Prototype in R or Python

Pleasant to work in. It runs.



Apply it to France, over a century

It runs for three weeks.



Have the core rewritten in C++

By someone else. Two codebases, two languages, two people — and the bugs live in the seam.

Julia's claim: **one language, with Python's flexibility and C's speed.** You do this on 21 October.

And the part that matters more than speed: in Julia, dependency management lives *inside the project*.

`Project.toml` and `Manifest.toml` travel with the folder. In R and Python it was bolted on afterwards. That is rung 4, by default.

WHY FIRS IMPOSES THREE

**You are not learning three
languages.
You are learning one set of
concepts
and three syntaxes.**

A variable is a variable everywhere. A loop is a loop everywhere.
Never say “I’m an R person” — it is not an identity, it is a
screwdriver.

I am not teaching you to write. I am teaching you to **read**.

By the end of this block you should be able to open a script you did not write and say: **that is a variable, that is a function, that is a loop** — and roughly see what is going on.

That is exactly the skill January asks of you for the model cartography, on model code that was not written for you and is not in your language.

Unit value of roundwood exports

R

```
# unit value of roundwood exports

value_usd  <- c(1250000, 890000, 2100000,
               445000, 1670000)
quantity_kg <- c(3200000, 2450000, 5100000,
               980000, 4300000)

unit_value <- function(value, quantity) {
  value / quantity * 1000 # USD per tonne
}

uv <- unit_value(value_usd, quantity_kg)

cat("Mean unit value:",
    round(mean(uv), 1), "USD/t\n")
```

PYTHON

```
# unit value of roundwood exports

value_usd  = [1250000, 890000, 2100000,
              445000, 1670000]
quantity_kg = [3200000, 2450000, 5100000,
              980000, 4300000]

def unit_value(value, quantity):
    return value / quantity * 1000 # USD per tonne

uv = [unit_value(v, q)
      for v, q in zip(value_usd, quantity_kg)]

print("Mean unit value:",
      round(sum(uv) / len(uv), 1), "USD/t")
```

DO NOT READ THE WORDS — LOOK AT THE SHAPE

Four movements, in the same order



	R	PYTHON
1 · INPUTS what goes in	<pre>value_usd <- c(1250000, ...)</pre>	<pre>value_usd = [1250000, ...]</pre>
2 · DEFINITION the operation, written once	<pre>unit_value <- function(value, quantity) { ... }</pre>	<pre>def unit_value(value, quantity): ...</pre>
3 · APPLICATION applied to the inputs	<pre>uv <- unit_value(value_usd, quantity_kg)</pre>	<pre>uv = [unit_value(v, q) for v, q in zip(...)]</pre>
4 · OUTPUT what comes out	<pre>cat("Mean unit value:", ...)</pre>	<pre>print("Mean unit value:", ...)</pre>

Syntax is what changes between languages. **Structure** is what carries across them.

Variable, assignment — and type

A variable is a label stuck onto a value

```
value_usd <- c(1250000, ...) # R  
value_usd = [1250000, ...] # Python
```

The name, the content, and the act of sticking the label on. R accepts `=` too; convention is `<-`, and you will see both.

Every value has a type

Decimal, integer, text, true/false. It sounds trivial. **Half the bugs you hit this year come from a type that is not what you assumed.**

The classic: your quantity column is read as **text**, because one decimal was written with a comma. You take the mean and it crashes — **or worse, it does not.**

Data structures

CONCEPT	R	PYTHON
A sequence of same-type values	<code>vector c(1, 2, 3)</code>	<code>list [1, 2, 3]</code>
A bag of different things	<code>list()</code>	<code>list / tuple</code>
Key to value mapping	<code>list(a = 1)</code>	<code>dictionary {"a": 1}</code>
A table of data	<code>data.frame</code>	<code>DataFrame (pandas)</code>

A **data frame** is a spreadsheet as seen by a programming language. Named columns, rows, one type per column. **Ninety per cent of what you do this year goes through that object.**

And one that will cost you an evening: in R the first element of a vector is number **1**. In Python it is number **0**.

Function, argument, return value

R

```
unit_value <- function(value, quantity) { ... }
```

PYTHON

```
def unit_value(value, quantity): ...
```

Function

A machine. Something goes in, something comes back. This one takes a value and a quantity and gives back a price per tonne.

Argument

What goes in — here **value** and **quantity**.
What comes back is the **return value**.

Why bother?

Because the day you discover the formula was wrong, you fix it **in one place**. That is the whole reason, and it is enough.

Installing and loading are two different actions

A package is code someone else wrote that you reuse. Two distinct actions — and confusing them is every beginner's first mistake.

R

```
install.packages("sf") # once, on the machine  
library(sf)           # every session
```

there is no package called 'sf'

Not installed. Download it.

PYTHON

```
pip install geopandas # once, in the terminal  
import geopandas      # every session
```

could not find function "st_read"

Installed, but not loaded. Add the **library()** line at the top.

Two messages, two causes, two fixes.

THE ONE REAL DIFFERENCE BETWEEN THE TWO COLUMNS

Loop, and vectorisation

```
for (i in seq_along(value_usd)) {  
  print(unit_value(value_usd[i], quantity_kg[i]))  # a loop: one at a time  
}  
  
uv <- unit_value(value_usd, quantity_kg)           # vectorised: all five at once
```

5

rows — nobody can tell the difference

3 s

vectorised, on three million iterations

20 min

the same work, as a loop. You will hit this wall in remote sensing.

Vectorisation is R's statistical heritage. In Python you get it back with **numpy** and **pandas**.

"Are functions faster than loops?" — No.



Functions are a correctness tool

One place to fix, one place to test, one name that says what the thing does. A call costs a little time, so a program full of tiny functions is very slightly **slower** — and much easier to trust.

Speed comes from vectorisation

`mean(x)` is fast because the loop inside it runs in compiled C. Swapping a **for** loop for **sapply()** gains you nothing: it is still an interpreted loop in a different coat.

The loop is rarely the problem

```
result <- c(result, new_value)
```

This copies the whole object on **every** iteration. Ten thousand iterations, ten thousand copies. Pre-allocate, and a **for** loop is perfectly respectable.

Write the loop first and make it **correct**. If it is too slow, look for a vectorised function that already does the job.

Eleven words

variable

type

vector

data.frame

dictionary

function

argument

return value

loop

vectorisation

package

You do not have to be able to write it.
You do have to be able to read it.

10H00

Fifteen minutes

Back at 10h15.



PART FOUR · 10H15 – 10H45

Editors and IDEs

You now know what a language is, and you can read a script. So: **what do you actually write it in** — and which of them is worth learning properly?



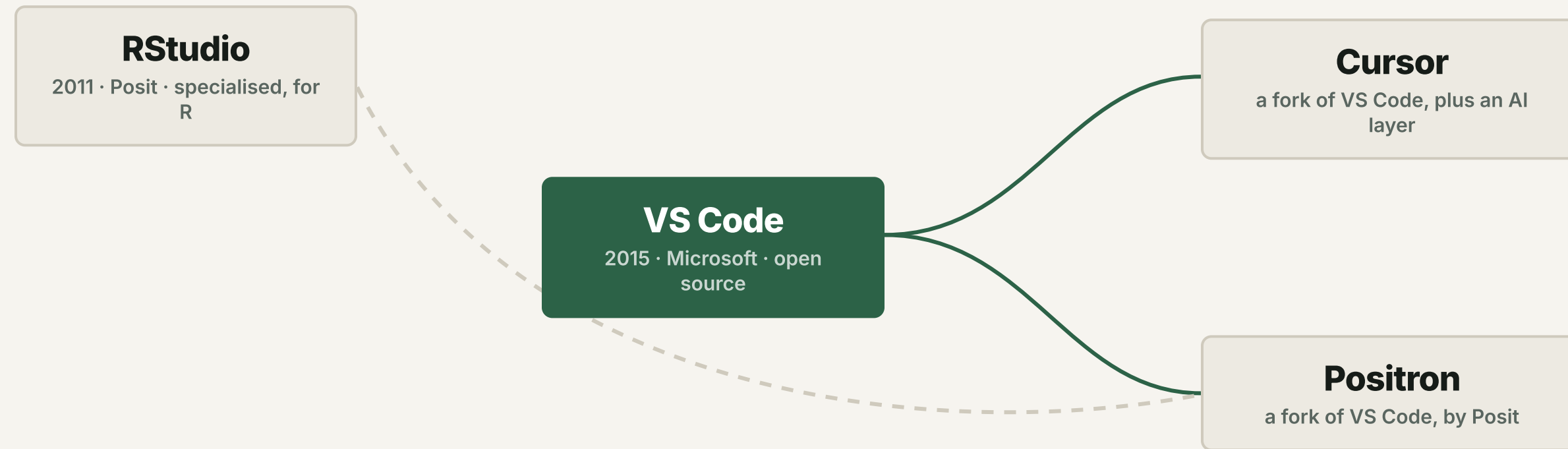
An IDE is a pane of glass, not the language

You can write R in Notepad and it will run.

An **integrated development environment** is simply the software you write code in. All it does is make your life bearable. It is not what executes your program.

No feature comparison table. The family tree explains everything.

Everything is converging on the same base



Learning VS Code means you already know how to use Cursor and Positron. The reverse is not true — and what you invest in VS Code will not be wasted.

What each one is actually for

RStudio

Four panes in one window: script, console, environment, plots.

The Environment pane is its best idea. A permanent list of everything that exists in your session — what makes programming *visible* instead of abstract.

Excellent for R. Adequate for Python.
Middling for the rest.

VS Code

On its own it does almost nothing. You add extensions and it becomes whatever you need — you installed three this weekend.

One tool for R, Python, Julia, Quarto, git, SQL and your configuration files.

The syllabus one, and the one that outlasts this school.

Cursor

A **fork**: someone took VS Code's source, copied it, built on top. Same extensions, same shortcuts, same settings.

Starting with Cursor without knowing the editor underneath means **never learning to debug** — you will never have had to read an error yourself.

What to watch, in each one

`demo-unit-value.R` and `demo-unit-value.py` — the programme from twenty minutes ago, as two real files.

RStudio	The Environment pane, while I run one line <code>value_usd</code> appears. The variable exists now, and you can see it. Then the project name, top right — that little thing solves the most annoying problem of the day.
VS Code	The explorer, the integrated terminal, the command palette The whole project folder on the left, not one file. Ctrl+` for a terminal in the same window. Ctrl+Shift+P — everything the software can do, found by typing three letters. And the <code>.R</code> and the <code>.py</code> open side by side.
Cursor	You cannot see a difference. That is expected. Then I select the function and ask it to explain itself.

The answer is correct. How do I know that?

Because I know what a unit value is, and I can see that dollars divided by kilos times a thousand gives dollars per tonne. **I was in a position to judge.**

The day you ask that about something you do not master — and it will happen this year — you are not. The answer arrives in **exactly the same confident tone** whether it is right or wrong.

These tools **accelerate someone who knows** and **mislead someone who does not**. Which is why the order matters: understand first, accelerate second.



RECOMMENDATION

Two tools. Not five.

RStudio — this term, for pure R

The Environment pane is worth a great deal when you are starting, and RStudio Projects solve a real problem. You use it at 13h50.

VS Code — the durable investment

Starting now. It is the syllabus one, it is the one for Quarto and git in October, and it is the one you keep.

And do not switch editors in the middle of a project because a classmate told you theirs was better.

PART FIVE · 10H45 – 11H00

The terminal

There is one thing all of those editors have in common, and it is **the oldest interface in computing**. Fifteen minutes, five commands, and one key.



A window where you type the name of a program instead of clicking its icon

That is all it is. Not hacking, not for initiates — the oldest interface there is, which survived because it is the most precise.

And you all opened one this weekend. When you typed `python --version`, you were in a terminal. It was not that bad.

Windows: Terminal or PowerShell. macOS: Applications › Utilities › Terminal. **And built in** — the Terminal tab in RStudio, **Ctrl+`** in VS Code. Most of the time you will not even open it separately.

Five gestures

GESTURE	COMMAND	
Where am I?	<code>pwd</code>	A terminal is always somewhere . In a folder. The most important sentence in this block.
What is in here?	<code>ls · dir on PowerShell</code>	The ground truth of what is in a folder — not what your file explorer has decided to show you. Hidden files included, with <code>ls -a</code> .
Go somewhere	<code>cd folder_name</code>	It moves the terminal , not you. Everything you type afterwards happens in there — which is why <code>pwd</code> comes first.
Go up one level	<code>cd ..</code>	Two dots means <i>the parent folder</i> . You meet it again in relative paths, in fifteen minutes.
Run a script	<code>Rscript x.R · python x.py</code>	Runs the file in a brand-new process , with nothing left over from before. Remember that — we come back to it at 11h40.

One key from this whole block: **TAB**. Three letters, then TAB, and the terminal completes the name. It prevents ninety per cent of typos, and a typo in a path costs you ten minutes. **Every time**.

THE ERROR EVERY ONE OF YOU WILL MEET

command not found

Typing **python** does not search your hard drive. The terminal looks in **a short list of folders known in advance**. That list is the **PATH**.

1 • The program is not installed

Install it.

2 • It is installed — but not on the PATH

Python was installed perfectly well. The terminal simply had no idea where to look.

This is why Friday's email said, in bold, to tick *Add python.exe to PATH*.

One message, two causes. **Knowing which one you have is ninety per cent of the fix** — and that sentence comes back three times today.

PART SIX · 11H00 – 11H30

The anatomy of a project

Thirty minutes here will save you dozens of hours this year.
I am not exaggerating.



Separate what comes in, what transforms, and what comes out

```
my-project/
├── data/
│   ├── raw/           <- inputs. NEVER modified.
│   └── processed/     <- what your scripts produce
├── src/               <- the scripts, prefixed 01_, 02_...
├── output/
│   ├── figures/
│   └── tables/        <- regenerable. Deletable.
├── config/
│   └── config.yml     <- parameters, never hard-coded
├── doc/              <- reports, notes, Quarto
├── README.md         <- what this is, how to run it
└── my-project.Rproj
```

WHY THIS ONE

Nothing original about it. It is roughly the same in every research group in the world — and that is exactly why it is worth using. Handed a colleague's folder that looks like this, you know where to look in ten seconds.

THIS IS THE TEMPLATE YOU DOWNLOAD DURING LABS

Lab 1 starts from precisely this tree, with eight scripts already in **src/**.

One folder is sacred. One folder is disposable.

data/raw/ is never touched

Not opened in Excel to fix a comma. Not overwritten. Not renamed. Everything you want done to it goes through a **script**, which reads from **raw/** and writes to **processed/**.

Nothing in output/ is precious

You should be able to delete the whole folder and rebuild it by running your scripts.

If deleting output/ frightens you, something is wrong — a result exists that no script can reproduce, and that result is not defensible.

The **README.md** is not decoration. It is what moves your project from **rung 2 to rung 3**: what this folder is for, what needs installing, what order to run things in. Four lines. Four lines your partner will not have to ask you for at half past two.

Three reasons why raw data is read-only

1

You can always start again

Get the cleaning wrong, rerun the script. The original is intact. Overwrite the file and it is gone — nobody can reconstruct it.

2

A fix in a script is written down

A fix made by hand in Excel is recorded nowhere. Six months later you will not know which of four thousand rows you touched, or why.

3

You must be able to walk back

The day a reviewer, an examiner or an agency asks where a number came from, you go from the result to the source with **no gaps**. Manual cleaning is a gap in that chain.

Excel modifies your data without asking

This
afternoon

Côte d'Ivoire comes back mangled

The accents are gone. Nothing warned you. Press Save at that point and **the damage is the file.**

This
afternoon

A twelve-digit trade value is handed back as 1.23E+11

2016

The gene SEPT1 becomes 1-Sep

A study found corrupted gene names in about **a fifth** of the genomics papers it checked. The naming committee eventually gave up and **renamed the genes** rather than keep fighting the spreadsheet.

That is where “just having a quick look in Excel” ends.



A script does not know where it is

It **believes** it is somewhere.
That somewhere is the **working directory**.

When a script says `read.csv("data/raw/comtrade.csv")` it is not searching your disk. It is looking **relative to its working directory**.

If the working directory is not what you assume, the file does not exist — even though you can see it sitting right there in your file explorer.

Absolute, and relative

ABSOLUTE — THE FULL ADDRESS, FROM THE ROOT OF THE DISK

```
read.csv( "C:/Users/valentin/Documents/thesis/project/data/raw/comtrade.csv" )
```

A **declaration**: this script will only ever run on my machine, under my username, until I reorganise my disk. Send it to your partner — it does not work. Put it on a server — it does not work. Tidy your **Documents** folder — it stops working even for you.

RELATIVE — THE ADDRESS FROM HERE

```
read.csv( "data/raw/comtrade.csv" )
```

And the worst version of the first one: `setwd("C:/Users/valentin/...")` **at the top of a script**. You will see it in scripts you are given, by serious people. It is the first line to delete.

Three answers, and the first one is free

The RStudio Project

```
# Open my-project.Rproj  
# The working directory IS  
# the project root.  
# There is nothing to write.
```

RStudio's best idea, and the one nobody notices. You never type `setwd`. Ever.

here(), in R

```
library(here)  
read.csv(here("data", "raw",  
              "comtrade.csv"))
```

Robust even outside a project. You install this afternoon.

pathlib, in Python

```
from pathlib import Path  
root = Path(__file__).parent.parent  
data = root / "data" / "raw" / "comtrade.csv"
```

Python has no **.Rproj**. Opening the **folder** in VS Code is the nearest thing.

Four traps



1 • Separators

Windows writes `\`, the rest of the world writes `/`. Good news: `/` works everywhere, including on Windows, in both languages.

Always write `/`.

And a technical reason never to write `\`: in a text string a backslash has a special meaning. In `"C:\data\new"` the machine reads `\n` as *newline*. Your path is broken and the error will not tell you why.

2 • Accents and spaces

`Données récoltées 2025 (final).csv` works on your machine, breaks on your partner's, breaks on a server, breaks in a URL.

An accent can be encoded differently on Mac and Windows, so the same name to your eye is **two different files** to the machine. No accents, no spaces, no brackets. Ever.

3 • Case

`Comtrade.csv` and `comtrade.csv` are the **same file** on Windows and **two different files** on macOS and Linux.

Your script works for you and breaks for your partner, over one capital letter.

4 • OneDrive, iCloud, Drive, Dropbox

These services **move your files while you are standing on them**. The file shows in the explorer, is no longer on the disk, and has to be re-downloaded — and your script does not know how to wait.

Keep your code projects outside the synced folder. From 20 October, git does that backup job far better.

Three rules for files, and one anti-pattern

1 No accents, no spaces

We have just covered why. Variables in **snake_case**: **unit_value**, not **UnitValue** and certainly not **uv2**. A name is meant to be read; it is not a password.

2 Dates as YYYY-MM-DD

2026-09-07, not **07-09-26** and definitely not **7 sept**. In that format **alphabetical order is chronological order**. Your files sort themselves and you did nothing.

3 Prefix scripts with their run order

01_import.R, **02_clean.R**, **03_analysis.R**. Whoever opens your folder knows the order without asking you. **That is rung 3 again.**

```
analysis.R  analysis_v2.R  analysis_final.R  analysis_final_v2.R  analysis_final_REALLY_final.R
```

Version control by hand. It does not work — and there is a tool that does it properly. It is called **git**, it is October, and you have already installed it.

PART SEVEN · 11H30 – 11H45

The anatomy of a script

You have a project that other people can open. Now, the file itself. Two things left: **what a well-written script looks like**, and one rule that will save you a great deal of embarrassment.



Six blocks, always in that order

R

```
# 02_clean.R – cleans the raw extract
# Input  : data/raw/comtrade_roundwood_2023.csv
# Output : data/processed/trade_clean.csv

# --- Dependencies ---
library(here)

# --- Parameters ---
MISSING_CODE <- -999 # unreported quantity
HS_LEVEL     <- 6    # 6-digit subheadings

# --- Functions ---
unit_value <- function(value, quantity) {
  value / quantity * 1000 # USD per tonne
}

# --- Body ---
trade <- read.csv(here("data", "raw",
                      "comtrade_roundwood_2023.csv"))

# --- Output ---
write.csv(clean, here("data", "processed",
                     "trade_clean.csv"), row.names = FALSE)
```

PYTHON

```
# 02_clean.py – cleans the raw extract
# Input  : data/raw/comtrade_roundwood_2023.csv
# Output : data/processed/trade_clean.csv

# --- Dependencies ---
from pathlib import Path
import pandas as pd

# --- Parameters ---
ROOT      = Path(__file__).parent.parent
MISSING_CODE = -999 # unreported quantity
HS_LEVEL    = 6    # 6-digit subheadings

# --- Functions ---
def unit_value(value, quantity):
    return value / quantity * 1000 # USD per tonne

# --- Body ---
trade = pd.read_csv(ROOT / "data" / "raw"
                   / "comtrade_roundwood_2023.csv")

# --- Output ---
clean.to_csv(ROOT / "data" / "processed"
             / "trade_clean.csv", index=False)
```

Functions go above the body, never inside it. One place to read what the script can do — and the layout of a script is not a language convention, it is a habit that travels with you.

Inside out. Never left to right.

```
write.csv(subset(trade, flow == "Export"), "output/tables/exports.csv")
```

1 • The innermost part

`subset(trade, flow == "Export")` — take the rows of `trade` where the flow is Export.



2 • Step out one level

`write.csv(..., "output/tables/exports.csv")` — write that result to that file.

Build the habit now. It is what lets you take apart the scripts you are handed.

Comment the **why**, not the **what**

Worthless

```
# divide by 1000
```

The line already says it. The comment adds nothing — and the day someone changes the line without changing the comment, **it becomes a lie.**

A false comment is worse than no comment.

Worth writing

```
# * 1000: quantities are in kg,  
#   unit values are per tonne  
# -999: Comtrade's missing code,  
#   not a real value
```

These say something the code does not. They record a **decision**, and its reason.

The test: **does this comment answer a question I would ask myself in six months?** If not, delete it.



A script is only correct if it runs from an empty session, top to bottom, with no intervention

You work for three hours. You run lines out of order, fix some, delete others. At the end it works — thanks to an object you created two hours ago with a line you have **since deleted**. It is still in the session's memory. It is no longer in the script.

You reopen tomorrow. Nothing works. And you cannot understand it, because yesterday it worked — and yesterday, it genuinely did.

In RStudio

Session > Restart R (Ctrl+Shift+F10), then **Ctrl+Shift+Enter** to run the whole file. **Two shortcuts. Learn them today.**

In VS Code

```
python src/02_clean.py
```

Running a script from the terminal starts a **brand-new process every time**. No leftover state, because there is no session left over to hold any. Restart-and-run-all by construction.

LAPTOPS OPEN. THIRTY SECONDS. DO IT NOW.

Tools › Global Options › General

- ✗ untick **Restore .RData into workspace at startup**
- ✗ set **Save workspace to .RData on exit** to **Never**

By default RStudio restores your last session. That is exactly what we do not want: it brings back ghost objects whose origin you do not know, and it makes you believe your script works.

This afternoon you get your hands on it

Lab 1

Build a clean project

From the template. At the end your partner takes your folder and runs it **without changing anything**. That is your move to rung 3, verified by someone other than you.

Lab 2

Read a deliberately filthy data file

Wrong encoding, semicolon separators, comma decimals, missing values, mixed units. In other words: a real export. Half the job of any engineer who works with data.

Lab 3

Repair five broken scripts

The **fifth one does not crash**. It gives you a wrong answer and says nothing. That is the most important of the five.

Everything is on github.com/vlmathieu/firs-uc33-modeling — and on the last page of your handbook. See you at 13h30.

FOR THE AFTERNOON — AND UNTIL JANUARY

The pairs

PAIR 1

Aitor Lopez
Louis-Félix de Crémiers

PAIR 2

Louise Gouget
Juliette Bessard

PAIR 3

Killian Mauferon
Bosco de Cacqueray

PAIR 4

Camille Pétard
Amédéo Taisne

PAIR 5

Antoine Corbeaux
Maxime Etrillard

PAIR 6

Alexandre Stona
Cécilia Debeaupuis

Built from the questionnaire, not at random. **Every pair mixes levels, and every pair mixes where you came from** — forestry and geomatics, on purpose. The same pairs this afternoon, and for the model cartography in January.

11H45 – 13H30 · LUNCH

Back at half past one

Bring the laptop you set up this weekend, and sit next to the person you have just been paired with.

If one of the three commands still fails on your machine, find me before 13h30 rather than at 13h50.



13H30 – 16H45

Three labs

Lab 1 — my project runs on your machine. Rung 3, verified by someone else.

Lab 2 — a file is not data until you have declared how to read it.

Lab 3 — the dangerous error is the one that does not crash.



WHERE EVERYTHING LIVES

github.com/vlmathieu/*firs-uc33-modeling*

One address. Not an email attachment, not a shared drive, not a PDF I send you in October.
Bookmark it.

1 • Nothing is ever renamed or moved

A link I hand you today still works in January. A rule I impose on myself — the same rule I gave you this morning about naming files.

2 • Folders numbered, sessions dated

00-admin, 02-reference, 04-labs/2026-09-07-lab1-... Alphabetical order is reading order. **This repository applies what it teaches** — check rather than believe me.

3 • There is a **CHANGELOG.md** at the root

When I correct something — and I will, this material is new — it is written there, dated. Working from something you downloaded three weeks ago? **That is the first file to open.**

The map

00-admin	The installation guide, and <code>how-to-ask-a-question.md</code> Three minutes to read.
02-reference	Glossary English–French · R / Python / Julia equivalents · terminal cheatsheet Half of you work in your second language, and “a data frame” is not a word anybody taught you.
04-labs	One folder per lab: a <code>brief.md</code> you follow, a <code>challenges.md</code> if you finish early
05-data	The two CSV files — and a <code>README</code> that is the data dictionary What each column means, what unit it is in. You need it in lab 2. Do not spend twenty minutes guessing.
06-cartography	Empty today. It is there so you can see it coming.

The solutions are not there yet. They go up tonight, fully commented.

WHEN YOU ARE STUCK — AND YOU WILL BE STUCK

Three doors, in this order

1 · Your partner

Always first. Six pairs in here, one of me.



2 · Me

Second.



3 · An issue on the repository

A public question that I answer publicly.

Why not email? Because **your problem is almost never yours alone**. Hit an encoding error on Windows this afternoon and two other people in this room have it, and four more will hit it in October.

An email answers one person once. An issue answers the whole cohort — and it is still there in January.

Six fields. Together they are a **reprex**.

Your operating system

Half of all errors are platform-specific, and it is the first thing anyone will ask.

Your version of R or Python

The other half.

What you were trying to do

The goal, in one sentence. Not the symptom.

The minimal code that reproduces it

Not your three hundred lines. The five that break.

The complete error message

As text, not a screenshot. In full — including the lines that look irrelevant. The cause is almost always in one of them.

What you have already tried

You will very often find the answer while filling it in. Not sometimes — often. Reducing your problem to five lines *is* debugging.

Four rules for the next three hours

1 Partner first, me second

Two people at one screen and I walk past. A hand up with an idle partner and I go to the partner.

2 Finished the core? Do not wait for me — open `challenges.md`

The challenges are not extra credit. They are what stops you being bored.

3 You will get errors. That is the exercise, not an accident.

Especially in lab 3, where all five scripts are broken on purpose, by me. **Read the message before you call me over. All of it, to the last line.**

4 Before you leave tonight, every pair opens one issue

A real question — or, better, a fault you found in this repository. Lab 1 challenge 5 asks you to audit it against the rules it imposes on you. Find one and you get to say so in class.

One constraint, not negotiable: this repository is **public**. No names, no marks, no personal data, no screenshot with an identifier in it.

Building a clean project

1–3 Get the template, install the data, open the project both ways

Together, front of room. Nobody runs ahead.

4 Fix `01_import.R` — it has three faults

An absolute path. A `setwd()`. An output written in the wrong place.

5 Install and load a package

6–7 Restart and run all · write the README

8 Swap folders with your partner

They may not change anything, and they may not ask a question. That is the only test that counts.

NO ARGUMENTS ABOUT THESE TWO

Put the project **outside** OneDrive, iCloud, Drive, Dropbox. And **do not open the CSV in Excel** — not once, not to have a look.

How you know step 6 worked: the script prints 361 rows. A different number is not a success — it is a filter you changed without meaning to.

There are **eight** scripts in `src/`, not two. Everything numbered **02, 03, 04** is lab 2. Leave them closed.

At 14h25 everybody swaps, finished or not. A half-finished project that does not run on the other machine has taught you more than a polished one nobody ever opened.

Installing and loading, for real this time

R — FROM INSIDE R

```
install.packages("here")      # once per machine
install.packages("ggplot2")   # you need it in lab 2

library(here)                 # every session
```

PYTHON — FROM OUTSIDE PYTHON

```
python -m pip install pandas  # in the terminal
py -m pip install pandas      # Windows fallback

import pandas as pd           # every session
```

PYTHON ONLY — THE THREE THINGS THAT GO WRONG ON THE WAY

Not at the >>> prompt

This goes in the **terminal**. If you see >>> in front of your cursor you are in the wrong place — type **exit()**.

python -m pip, not bare pip

Your machine has more than one Python, whether you know it or not. Bare **pip** may install into one while VS Code runs another — and the install still tells you it succeeded.

Then start a new session

A package installed while a session is open is invisible to that session. Ten minutes convinced it did not work.

R installs its packages from inside R. Python installs its packages from outside Python. That is why RStudio can offer you a button and VS Code cannot.

Dirty data

This file was **not rigged**.

It is the same Comtrade data as this morning's, exported once through a French-configured Excel. That is all that happened to it — and that is exactly the file a colleague sends you in March.

Steps 1 to 5 — in the console

You are exploring: you look, you print, you throw away. And step 1 is **look at the first three lines from the terminal** and answer four questions before typing any R or Python.

```
head -3 data/raw/..._dirty.csv  
Get-Content -Head 3 data\raw\..._dirty.csv
```

Step 6 — in a script

src/02_cleaning.R or **.py**, already in your project. That is the moment you stop exploring and start **recording decisions**.

A decision that lives only in a console is a decision nobody can check in six months — including you.

Checkpoint at step 3: 674 rows × 13 columns. Not both numbers? You are not loaded. Do not go on.



Same file. Two different complaints.

R

```
more columns than column names
```

R reached the column count before it reached the accents.

Python

```
UnicodeDecodeError: 'utf-8' codec  
can't decode byte 0xb3
```

Python fell over the accents before it ever reached the columns.

An error message names the problem the tool hit **first** — not the list of everything that is wrong. Both files are equally broken; the two tools walked into it in a different order.

Remember that in lab 3. Fix an error, get a new one, and you have not made things worse — you removed the first obstacle and can now see the second.

15H00 · FIFTEEN MINUTES

One thing to take with you

Step 5 printed the distinct values of `partnerDesc`. Go and read that list properly before you leave — all of it.

There is something in that list that is not a country. Find it.



World

Not a partner. The **aggregate of all partners** — Comtrade puts the total in the table as a row, alongside the components it is the total of. Sum **primaryValue** without excluding it and you are adding the total to the parts.

\$1,557.0M

France total, with Worlḁ — if you get this, re-read your filter

\$778.5M

France total, without. Exactly half.

And nothing will tell you. No error, no warning, no red text. Your table loads, your `sum()` returns a number, your figure draws beautifully — and it is twice too tall. You would put it in a report. You would defend it in a viva.



Prefer the code to the label

Works today

```
partnerDesc != "World"
```

A **label** gets translated, renamed, abbreviated, spelled three ways and capitalised differently.

Use this one

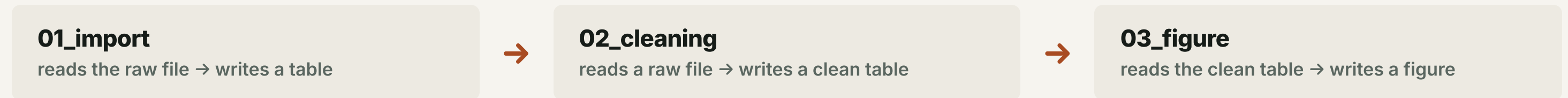
```
partnerISO != "W00"
```

A **reserved code** does not.

A reflex worth acquiring now: you meet a dozen classifications this year — HS codes, IGN species codes, NUTS regions — and **every one of them has stable codes and unstable labels.**

Keep that in your head. You meet it again in forty minutes, and I will not be warning you next time.

Clean, and a figure — but the pipeline is the lesson



Nothing in 03 knows that 02 exists. It reads a file off the disk, like a stranger would. That is why you can change a colour without re-cleaning six hundred rows, why your partner can rewrite the figure without opening your cleaning code, and why, when something breaks, **there is exactly one script to look at.**

The numbers in the filenames are the run order. That is this morning's naming rule doing actual work.

The brief gives you the line for each of the five decisions, so the code is not the exercise. **Writing down why is the exercise.** There is no right answer to any of them — there is a documented answer and an undocumented one, and only one is defensible in six months.

Three kinds of wrong

Lab 1 Your script broke — and you knew, because it broke

Lab 2 **A file refused to load. Annoying, noisy — and honest**
It told you it was wrong.

Lab 3 **Then you found *World*, which told you nothing at all**
An hour on that second kind. Five scripts. Four of them shout at you. **The fifth does not, and it is the only one that matters.**

Five scripts. None does what it claims.

SET-UP FIRST — THREE CHECKS, TWENTY SECONDS

```
list.files("src/broken-scripts")  # ten files
list.files("data/raw")           # two csv files
getwd()                          # ends in uc33-lab1
```

The ten scripts go in a **new** folder, **src/broken-scripts/** — not loose beside your own **01** to **04**, which are a pipeline. Two meanings for one numbering in one folder is the mess we spent this morning avoiding.

They read **data/raw/...** **relative to the project root**. Run them from anywhere else and all five fail identically — and script 1 stops being an exercise.

HOW TO SPEND THE HOUR

5'	each, for scripts 1 to 3
15'	script 4
20'	script 5 — worth the other four put together

Still on script 2 at four o'clock? Skip to script 5. This is not a race, and I will be asking "why" at every table.

The four-step method — and four things on paper



- 1

Read the error

In full, to the last line. An error message is **information**, not punishment. It almost always contains the name of the thing that broke and the line it broke on.
- 2

Read the documentation

`?read.csv · help(pd.read_csv)`
- 3

Search — stripping out what is specific to you

Your filenames, your variable names. The message that gets zero results with your username in it gets ten thousand without.
- 4

Ask. Partner, then me.

Entirely legitimate. It just comes after the other three.

FOR EVERY SCRIPT, ON PAPER

Symptom	what the machine printed
Diagnosis	what was wrong, in one sentence
Fix	what you changed
Prevention	what would have stopped you writing this bug in the first place

That fourth column is the one that serves you all year. The first three you will have forgotten by Thursday.

The decimal mark was never declared, so the column came in as **text**

R refuses

```
invalid 'type' (character) of argument
```

It will not add up text. It stops.

Python does it

```
no error
```

sum() on a text column **concatenates the strings** and hands you back a “number” six thousand three hundred and forty-nine characters long. The program finishes normally and reports success.

A more permissive language is **not** a safer language.

This one does not crash

French oak log exports, 2022–2024:
909.7 million USD
104 reported flows

It runs cleanly. The number is wrong. It is **exactly twice** the right answer — and three rows out of a hundred and four are enough to do that.

- 1

Find out why, without being told where to look
- 2

Fix it
- 3

How would you have noticed if I had not warned you?
That is the one I am actually asking for.



How you would have caught it

1 · Order of magnitude

900 million dollars of oak logs over three years, against a French oak harvest of roughly two million cubic metres a year, implies a price well above anything the market pays. 450 is plausible.

Knowing what the answer should look like before you compute it is the only real protection — and it is not a coding skill. It is knowing your subject.

2 · The internal check

The sum of the parts must equal the whole. The **World** row *is* the whole. It was in that file to serve as a check — and it was used as a term instead.

3 · Look before you aggregate

The distinct values of a column, before summing it. Thirty seconds. That is all it costs, every single time.

```
stopifnot(!"World" %in% oak$partnerDesc)
```

One line. It turns a silent failure into a loud one. **That is the entire definition of a test**, and it is the whole of October.

Where you got to

4

A stranger, three years from now, gets exactly my numbers

20 Oct git, the history · 21 Oct environments, so “here was not installed on the other machine” has an answer · 23 Oct tests, so your project checks the assertion for you

OCTOBER



3

It runs on my partner's machine

Not something you can claim — somebody else verified it for you this afternoon. For some of you the answer was no. That is what the test was for.

ALL THE LABS

2

It runs on my machine, six months from now

Relative paths, the .Rproj, the 01_02_ prefixes, the README.

LAB 1 ✓

1

It runs on my machine, right now

BEFORE LUNCH ✓

IF YOU KEEP ONE THING FROM TODAY — AND IT IS NOT ABOUT CODE

**The dangerous error is not
the one that crashes.
It is the one that produces a
perfectly presentable number.**

Everything we do between now and January is about making the
second kind behave like the first.

Tonight, and 20 October

1 The solutions to all three labs go up on the repository tonight

Fully commented, with the reasoning behind every decision — including one bug I made myself while writing them, which I have left in with an explanation, because it is the best example in the whole set. **Read them tomorrow morning against what you actually did.**

2 00-admin/how-to-ask-a-question.md

Three minutes. Tonight.

3 Every pair opens one issue before tomorrow

A real question, or a fault you found in my repository. Lab 1, challenge 5 tells you where to look.

| Next session is **20 October** — and you will already know why you need git.

